

From Graphical Models to Generative AI

Representations → Constraints → Dynamic Programming

Michael (Misha) Chertkov

MATH 577-001 University of Arizona

Lecture 3 — Monday, August 31, 2026

Mathematics 514

Today: 3-minute research lightning talks, then Chapter 2 representations → Chapter 3 computation.

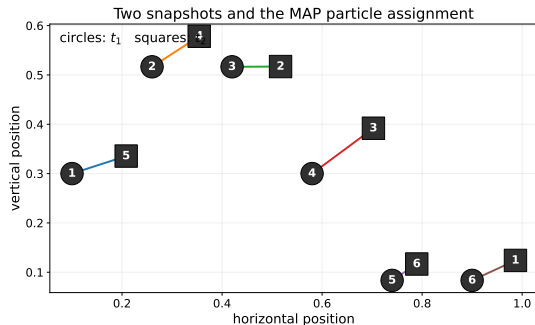
Three research neighborhoods — detailed guide next lecture

Continuous Structured GenAI	Quantum / Stat Phys / Information	Optimization / Networked Systems
scientific / continuous generation	quantum, spins, inference, tensors	optimization, networks, constraints

I am preparing a short studio guide: scope, book map, suggested papers, and Project Card guidance. We will start Lecture 4 with it.

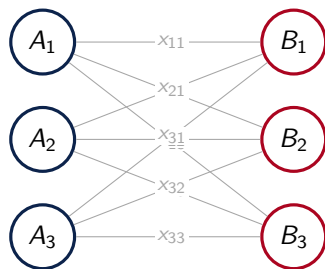
Individual projects; shared mathematical tools; cross-studio interaction is encouraged. Project Card due Sep. 11.

A reverse representation example: particle matching starts naturally as an NFG



Two snapshots; an unknown one-to-one association.

NFG $\longrightarrow$ FG: replace every edge variable x_{ab} by a variable node.



factors at vertices; variables on edges

$$p(\mathbf{x}) \propto \prod_a A_a(\mathbf{x}_{a*}) \prod_b B_b(\mathbf{x}_{*b}).$$

Hard constraint versus soft penalty — and a careful AI-era analogy

Hard factor

$$f_{\text{hard}}(x) = \mathbf{1}\{c(x) = 0\}$$

Violations have exactly zero probability.

Soft factor

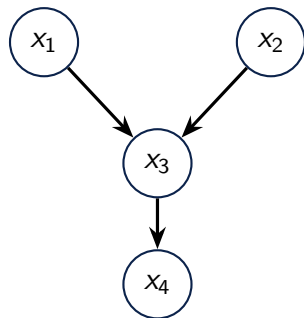
$$f_{\text{soft}}(x) = e^{-\lambda c(x)}$$

Violations are penalized, not forbidden.

AI analogy: learned loss/reward shapes behavior; a verifier or explicit checker can enforce structure more directly. But a “verifiable reward” is still a reward unless invalid outputs are actually rejected/projected.

Recent context: DeepSeek-R1 (2025, rule-based accuracy/format rewards); reward-hacking studies emphasize that the verifier itself can be exploited.

Directed representation: Bayesian network = conditional factorization



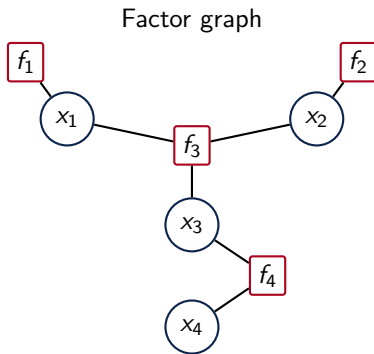
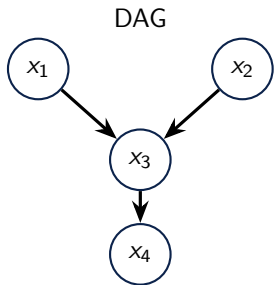
A DAG / Bayesian network.

$$\begin{aligned} p(\mathbf{x}) &= \prod_i p(x_i \mid x_{\text{pa}(i)}) \\ &= p(x_1)p(x_2)p(x_3 \mid x_1, x_2)p(x_4 \mid x_3). \end{aligned}$$

- ▶ every node contributes one **conditional factor**;
- ▶ its scope is the family $\{i\} \cup \text{pa}(i)$;
- ▶ direction gives a normalized generative factorization.

Arrows encode a directed factorization; they are not automatically causal without additional assumptions.

DAG $\rightarrow$ factor graph: use the conditional factors directly

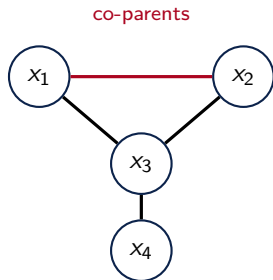


$$\begin{aligned} f_1 &= p(x_1), & f_2 &= p(x_2), \\ f_3 &= p(x_3 \mid x_1, x_2), & f_4 &= p(x_4 \mid x_3), \end{aligned} \quad p(\mathbf{x}) = f_1 f_2 f_3 f_4.$$

No moralization and no d -separation are needed: expose the scopes of the conditional factors.

Factor scopes $\rightarrow$ moral/primal graph $\rightarrow$ clique representation

Moral graph = primal graph of BN
family factors



For a BN: **marry co-parents, then drop arrowheads**. Equivalently: connect every pair of variables appearing in a common family factor.

Maximal cliques here are

$$K_1 = \{1, 2, 3\}, \quad K_2 = \{3, 4\}.$$

They may **overlap**; they are not independent pieces.

$$p(\mathbf{x}) = \frac{1}{Z} \prod_{K \in \mathcal{C}_{\max}} \phi_K(x_K).$$

Original factors can be absorbed into any containing clique. For this BN, $Z = 1$ if we retain the normalized conditional factors.

Hammersley–Clifford

For $p > 0$: Markov w.r.t. an undirected graph $G \iff$ factorization over the cliques of G .

d -separation and the road to junction trees

CI query in a DAG

For $A \perp B \mid C$:

1. keep ancestors of $A \cup B \cup C$;
2. moralize that ancestral DAG;
3. delete C ;
4. test ordinary graph separation.

For the collider $x_1 \rightarrow x_3 \leftarrow x_2$:

$$x_1 \perp x_2, \quad x_1 \not\perp x_2 \mid x_3.$$

This is a theorem for reading CI from the DAG — not a step in building its factor graph.

Factor graph: factorization.

d -separation: CI.

Junction tree: exact inference.

Exact-inference route — details later

DAG / factor scopes



moral (primal) graph



triangulate / add fill-in

maximal cliques



junction tree

Query-specific pruning may come first, but constructing a junction tree does not require a d -separation pass.

Dynamic programming = distribute the sum before the product becomes global

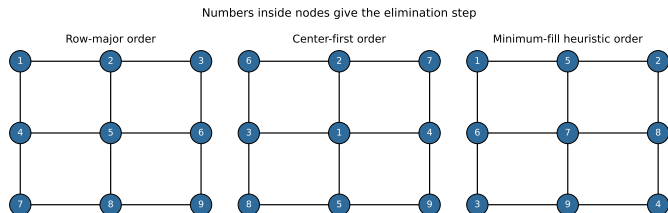
$$Z = \sum_{x_1, x_2, x_3, x_4} \psi_1(x_1) \psi_{12}(x_1, x_2) \psi_2(x_2) \psi_{23}(x_2, x_3) \psi_3(x_3) \psi_{34}(x_3, x_4) \psi_4(x_4).$$

collect $\rightarrow$ combine $\rightarrow$ eliminate $\rightarrow$ replace

$$m_{1 \rightarrow 2}(x_2) = \sum_{x_1} \psi_1(x_1) \psi_{12}(x_1, x_2).$$

The intermediate object is a function on the separator. On a tree, leaf elimination creates no fill-in.

Why trees are special — and what changes away from trees



- ▶ Tree: remove a leaf $\Rightarrow$ one-variable separator.
- ▶ Loop/grid: elimination can connect surviving neighbors.
- ▶ Those new computational edges are **fill-in**.
- ▶ Same exact answer; very different intermediate complexity.

Notebook for a quick look: `ch03/ch03_01_elimination_order_and_treewidth.ipynb`